WEEK 4 TASK REPORT

Predictive Modeling and Optimization in Logistics Systems

Delivery-time forecasting with machine learning and data-driven dispatch, alerting, vehicle-allocation and route-planning optimization

DATA DISCLOSURE

All data used in this project is **simulated** (synthetically generated by the Python code in the accompanying repository). It is **not** real company data, and no real organisation, customer, driver or vehicle is represented. Every result, including the cost and time savings in the optimisation section, is a result on this simulated system and should be read as a demonstration of the method, not as a measured saving for any real business.

Prepared by **Shruti Rani**

Contents

Abstract
1. Introduction and Problem Definition
2. Dataset (Simulated)
3. Methodology and Model Selection
4. Evaluation Metrics and Validation Strategy
5. Results and Model Comparison
6. Optimization Strategies
7. Recommendations and Implementation Roadmap
8. Limitations and Threats to Validity
9. Reproducibility and Code Verification
10. Conclusion
References
Appendix A: Repository Structure
Appendix B: Tuned Hyper-parameters

Abstract

Late and unpredictable deliveries inflate cost, hurt customer trust and waste fleet capacity. This project builds a machine-learning model that forecasts **dispatch-to-delivery time (minutes)** for last-mile deliveries and then uses the forecasts to drive four operational decisions: dispatch timing, traffic-risk alerts, vehicle allocation and route sequencing.

Because real operational data was not available, a **simulated dataset of 6,000 deliveries** was generated with a documented data-generating process (distance, stops, vehicle type, weather, congestion, hour of day and more, plus random noise and rare incidents). Six models were compared with 5-fold cross-validation: a mean baseline, linear regression, ridge regression, a decision tree, a random forest and gradient boosting. Four were tuned by randomized search. Models were compared with **MAE, RMSE and R^2**, and the winner was chosen on cross-validation RMSE only; the hold-out test set was used once.

Gradient Boosting performed best on the hold-out test set with **MAE = 4.15 min, RMSE = 6.18 min and R^2 = 0.897**, compared with an MAE of 9.92 min for a rule-of-thumb ETA. For reference, a perfect predictor of the simulator's expected time would still show RMSE = 5.78 min because of irreducible noise, so the model is close to the achievable ceiling on this data. Using the predictions, a risk-alert scheme flagged 92.3% of late deliveries in advance, a model-guided vehicle allocation cut simulated cost by 5.4% and late parcels by 42.4% versus a rule-based dispatcher, and shifting flexible peak-hour deliveries reduced their on-road minutes by 3.6%. A negative finding is also reported honestly: traffic-aware route sequencing added almost nothing beyond a good 2-opt optimiser in this setting.

1. Introduction and Problem Definition

1.1 Logistics context

In last-mile delivery the customer is quoted an estimated delivery time at dispatch. Most operators still quote it with simple rules such as a fixed handling time plus minutes per kilometre. Those rules ignore congestion, weather and vehicle type, so they are right on quiet clear days and wrong precisely when the operation is under stress. Late deliveries then trigger penalties, re-deliveries and support calls, while pessimistic quotes waste vehicle time.

1.2 Prediction problem

Item	Definition
Task type	Supervised regression
Target variable	`delivery_time_min`: minutes from vehicle dispatch to the parcel being delivered
Prediction moment	At dispatch time. Only information available at that moment is used as input (no post-delivery data, so no target leakage)
Unit of analysis	One delivery (one parcel on one vehicle run)
Business use	Better ETAs, early warning of likely-late deliveries, and inputs to dispatch / allocation / routing decisions

The rule-of-thumb ETA that the business currently quotes (`promised_eta_min` = 14 + 3.0 × km + 2.0 × (stops − 1)) is deliberately **excluded from the model's features**. It serves as the business baseline for comparison: 21.1% of deliveries in the full simulated dataset exceed it.

1.3 Objectives

1. Build and validate a predictive model for delivery time, comparing simple and advanced techniques fairly.
2. Explain why MAE, RMSE and R^2 were chosen and how the models were compared.
3. Convert predictions into practical decisions: dispatch planning, traffic-risk alerts, vehicle allocation and route-planning improvements.
4. Make the entire study reproducible, with a script that re-runs everything and checks that the results match.

1.4 Flow of the study

The report follows the workflow of the code: **data simulation → preprocessing → model comparison with cross-validation → tuning → hold-out evaluation → optimisation modules → recommendations**. Code excerpts are shown next to the corresponding analysis; the complete code is in the repository.

2. Dataset (Simulated)

> **Dataset transparency**
> The dataset is simulated. It was produced by `src/simulator.py` with a fixed random seed (42). The relationships inside it are assumptions based on common logistics knowledge, not

measurements from a real fleet. Because the model is trained on data whose structure is known, its accuracy here is likely **higher than it would be on messy real-world data**.

2.1 Why simulate and how the data was generated

No operational dataset was available for this assignment, and real delivery data is usually confidential. A transparent simulator also has a research advantage: because the true expected delivery time is known, optimisation decisions can be scored against ground truth rather than against the model's own predictions (Section 6). The generating process combines the following effects:

- **Driving time** = distance ÷ vehicle speed (bike 16, van 28, truck 22 km/h) × a non-linear congestion factor × a weather factor × small driver-experience and load effects.
- **Congestion** follows a daily profile with a morning peak near 09:00 and an evening peak near 18:00, is higher in the central zone, in bad weather and on weekdays, and is clipped to a 0–10 index.
- **Fixed components**: loading time and handling time that grow with parcel weight, and waiting time that grows with the number of stops on the run.
- **Weather**: bikes are hurt more by rain and storms than vans or trucks; congestion falls less for bikes.
- **Noise**: multiplicative log-normal noise (σ = 0.10) plus a 3% chance of an unpredictable incident adding 8–30 minutes.
- **Dispatch practice**: small, short-distance parcels may go by bike or van; heavy or long ones go by van or truck, mimicking present-day behaviour.

2.2 Features and target

Feature	Type	Description
distance_km	numeric	Route distance to the delivery point (0.5–40 km)
package_weight_kg	numeric	Parcel weight (0.1–300 kg)
stops_on_route	numeric	Number of drops on the vehicle's run (1–14)
hour_of_day	numeric	Dispatch hour (07–20)
day_of_week	numeric	0 = Monday … 6 = Sunday
traffic_index	numeric	Congestion level at dispatch, 0 (free flow) to 10 (gridlock); ~1.5% missing
driver_experience_years	numeric	Driver experience (0.5–20 years); ~1.4% missing
vehicle_load_pct	numeric	Vehicle load factor (20–100%)
vehicle_type	categorical	bike, van, truck
weather	categorical	clear, rain, fog, storm
zone	categorical	Central, North, South, East, West
priority	categorical	standard or express
delivery_time_min	**target**	**Minutes from dispatch to delivery**

2.3 Dataset summary

Statistic	Value
Rows / columns	6,000 / 16

Statistic	Value
Period covered (simulated)	2025-01-01 to 2025-12-31
Target mean / median / std	42.2 / 38.3 / 19.7 min
Target min / max	7.4 / 169.8 min
Vehicle mix	van 66.7%, bike 31.9%, truck 1.4%
Weather mix	clear 69.3%, rain 22.8%, fog 5.0%, storm 2.9%
Missing values	traffic_index: 99; driver_experience_years: 85
Late rate vs rule-of-thumb ETA	21.1%

Missing values were injected on purpose (about 1.5% in two columns) to mimic sensor or data-entry gaps and to demonstrate a realistic preprocessing step. They are handled by median imputation inside the modelling pipeline.

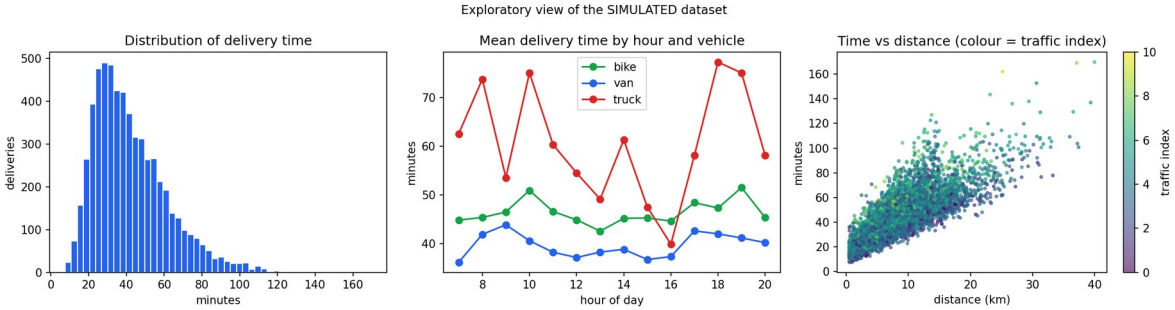


Figure 1. Exploratory view of the simulated dataset: target distribution, daily pattern by vehicle, and time versus distance.

Three patterns stand out. Delivery time is right-skewed, with a long tail of slow deliveries. Mean time rises in the morning and evening peaks. Time grows with distance, and the scatter around the trend widens with congestion, which signals an interaction between distance and traffic that a linear model cannot fully capture.

3. Methodology and Model Selection

3.1 Pipeline overview

5. **Split**: 80% training (4,800 deliveries) and 20% hold-out test (1,200), random with a fixed seed.
6. **Preprocess inside a Pipeline**: median imputation and scaling for numeric inputs; most-frequent imputation and one-hot encoding for categorical inputs. Fitting these steps inside each cross-validation fold prevents information leaking from validation data.
7. **Feature engineering**: `is_peak_hour` (08–10 and 17–19), `is_weekend`, and the interaction `dist_x_traffic`.
8. **Compare six models** with 5-fold cross-validation on the training set.
9. **Tune** four models by randomized search; **select** the best on cross-validation RMSE.
10. **Evaluate once** on the hold-out test set; run robustness checks (time-ordered split, error by segment).

Code 1. Preprocessing and model pipeline (src/train_models.py)

```
def build_pipeline(estimator):
    numeric = Pipeline([('impute', SimpleImputer(strategy='median')),
```

```
                    ('scale', StandardScaler())])
    categorical = Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                            ('onehot', OneHotEncoder(handle_unknown='ignore'))])
    prep = ColumnTransformer([('num', numeric, NUMERIC_ALL),
                              ('cat', categorical, CATEGORICAL)])
    return Pipeline([('features', FunctionTransformer(add_features)),
                     ('prep', prep), ('model', estimator)])
```

3.2 Model candidates and rationale

The candidates were chosen to span a range of complexity, so the added value of each step up in flexibility can be measured rather than assumed.

Model	Why it was included	Expected strength / weakness
Mean baseline	Floor: predicts the training mean for everyone. Any useful model must beat it.	$R^2 \approx 0$ by construction
Linear regression	Transparent baseline; coefficients are easy to explain to operations staff.	Fast, interpretable; cannot model non-linear congestion or interactions without manual features
Ridge regression	Linear model with L2 penalty; tests whether regularisation helps with correlated inputs (hour, traffic, zone).	Stable with correlated features; still linear
Decision tree	Captures non-linearity and interactions with human-readable rules.	Easy to read but a single tree has high variance and overfits
Random forest	Averages many de-correlated trees to reduce variance (Breiman, 2001).	Robust, little tuning; can be less sharp than boosting and larger to store
Gradient boosting	Builds trees sequentially, each correcting the previous errors (Friedman, 2001); a strong default for tabular data.	Usually most accurate on tabular data; needs tuning and is harder to interpret

4. Evaluation Metrics and Validation Strategy

4.1 Why MAE, RMSE and R^2

No single metric tells the whole story, so three complementary metrics are reported. With y the actual time, ŷ the prediction and n the number of deliveries:

Metric	Formula	Why it is used here
MAE	$\text{mean}(\lvert y - \hat{y} \rvert)$	Reported in minutes, so it reads directly as "the ETA is typically off by X minutes". It treats every minute of error equally and is robust to a few extreme outliers. It is the headline metric for ETA quality.
RMSE	$\sqrt{\text{mean}((y - \hat{y})^2)}$	Squaring penalises large misses more heavily. A 30-minute miss hurts the customer far more than three 10-minute ones, so RMSE reflects the cost of bad surprises. It was the metric used for model selection and tuning. A large RMSE/MAE ratio signals heavy-tailed errors.
R^2	$1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$	Unit-free share of variation in delivery time that the model explains (1 = perfect, 0 = no better than the mean). It lets us compare against the mean baseline and judge the model without needing to know the scale of minutes.

Using all three guards against misleading conclusions: two models can have a similar MAE yet very different RMSE if one makes occasional large errors, and R^2 shows how much better than a naive guess both are.

4.2 How the models were compared

- **Same data, same folds**: every model was scored on identical 5-fold splits (shuffled, seed 42) of the training set, so differences come from the model and not from luck in the split.
- **Mean ± spread**: cross-validated RMSE is reported with its standard deviation across folds, to see whether a gap between models is larger than fold-to-fold noise.
- **Fair tuning**: the four tunable models received the same budget (randomized search, up to 8 candidates, 3-fold inner CV). Linear regression has no hyper-parameters; the baseline needs none.
- **Selection rule**: the winner was picked on cross-validation RMSE **only**. The hold-out test set was untouched until after selection, so the final test numbers are an unbiased estimate.
- **Sanity checks**: comparison with the mean baseline, a time-ordered split, and error analysis by weather, vehicle and peak/off-peak.

Code 2. Cross-validation and hyper-parameter search (src/train_models.py)

```
cv        = KFold(n_splits=5, shuffle=True, random_state=SEED)  # reporting/selection
search_cv = KFold(n_splits=3, shuffle=True, random_state=SEED)  # inside the search
scoring   = {'MAE': 'neg_mean_absolute_error',
             'RMSE': 'neg_root_mean_squared_error', 'R2': 'r2'}

# Stage 1: default models, 5-fold CV
s = cross_validate(build_pipeline(est), X_train, y_train, cv=cv, scoring=scoring)

# Stage 2: randomized hyper-parameter search (Bergstra & Bengio, 2012)
rs = RandomizedSearchCV(build_pipeline(est), grid, n_iter=8, cv=search_cv,
                        scoring='neg_root_mean_squared_error', random_state=SEED)
rs.fit(X_train, y_train)

# Stage 3: choose by CV RMSE, then ONE evaluation on the hold-out test set
pred = best_pipeline.predict(X_test)
mae, rmse, r2 = mean_absolute_error(y_test, pred), \
                np.sqrt(mean_squared_error(y_test, pred)), r2_score(y_test, pred)
```

5. Results and Model Comparison

5.1 Cross-validation on the training set

Table 1. 5-fold cross-validation results (training data, default settings; last column after tuning)

Model	CV MAE (min)	CV RMSE (min) ± std	CV R^2	Tuned CV RMSE
Mean baseline	15.43	19.78 ± 0.31	-0.000	n/a (no tuning)
Linear Regression	5.20	7.34 ± 0.16	0.862	n/a (no tuning)
Ridge Regression	5.20	7.34 ± 0.16	0.862	7.34
Decision Tree	5.74	8.21 ± 0.35	0.827	7.80
Random Forest	4.59	6.68 ± 0.26	0.886	6.55
Gradient Boosting	4.23	6.18 ± 0.18	0.902	6.11

5.2 Hold-out test performance

Table 2. Final test-set performance (1,200 unseen deliveries; models tuned where applicable)

Model	Test MAE (min)	Test RMSE (min)	Test R^2
Mean baseline	15.28	19.27	-0.000
Linear Regression	5.12	7.16	0.862
Ridge Regression	5.11	7.16	0.862
Decision Tree	5.60	8.06	0.825
Random Forest	4.55	6.62	0.882
Gradient Boosting	4.15	6.18	0.897

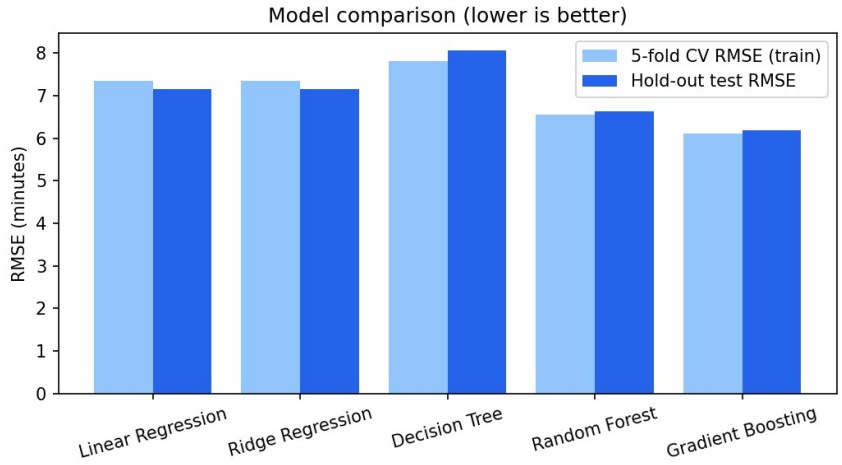


Figure 2. Cross-validated versus hold-out RMSE for each model. Close agreement between the two bars indicates no overfitting to the training data.

What the comparison shows

- **Everything beats the baseline.** The mean predictor reaches RMSE 19.27 min with $R^2 \approx 0$, so the features carry real signal.
- **Linear regression is a strong baseline** (R^2 0.862), which is expected because delivery time is largely driven by distance. Ridge gave the same result (0.862), i.e. regularisation was not needed with only 12 inputs.
- **A single decision tree is worse than linear regression** (RMSE 8.06 vs 7.16), a classic sign of high variance: one deep tree memorises noise.
- **Ensembles win.** Random forest improves to RMSE 6.62; gradient boosting is best at 6.18. Relative to linear regression, boosting lowers test RMSE by 13.7% and MAE by 18.9%. The gain comes from modelling the non-linear congestion effect and its interaction with distance and vehicle type.
- **No overfitting signal.** Cross-validated and hold-out errors are close for every model (e.g. boosting: CV RMSE 6.11 vs test 6.18), and the fold-to-fold standard deviation is small (about 0.18 min).

5.3 Selected model

Gradient Boosting was selected on cross-validation RMSE. Tuned hyper-parameters: `subsample=0.7`, `n_estimators=350`, `min_samples_leaf=10`, `max_depth=4`, `learning_rate=0.05`. On the hold-out test set it achieves:

Metric	Value	Interpretation
MAE	4.15 min	Typical error of about 4 minutes on a mean delivery time of 42.2 min
RMSE	6.18 min	RMSE/MAE = 1.49: errors are mildly heavy-tailed, driven by unpredictable incidents
R^2	0.897	The model explains about 90% of the variation in delivery time
Rule-of-thumb ETA MAE	9.92 min	The model cuts typical ETA error by 58.2%

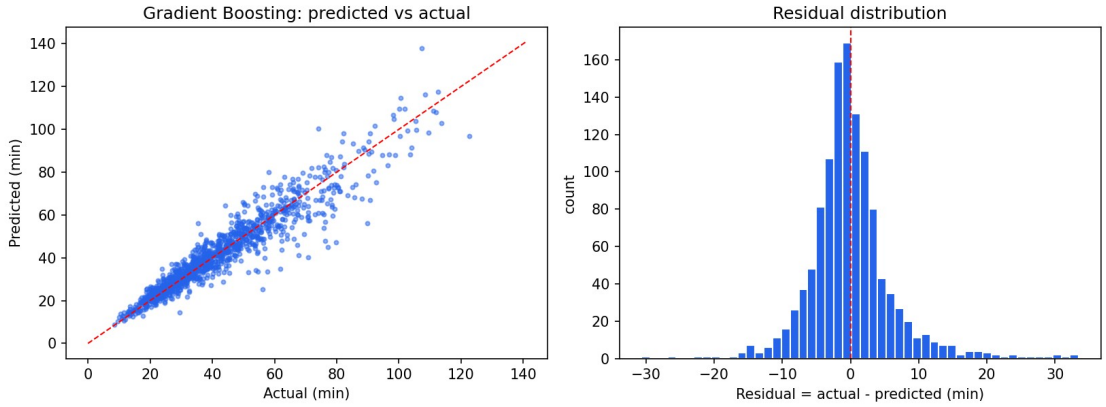


Figure 3. Predicted versus actual delivery time (left) and residual distribution (right) on the hold-out test set.

Points hug the diagonal across the whole range, and residuals are centred on zero with a slight right tail. That tail corresponds to the simulated random incidents (3% of deliveries), which no model can predict from dispatch-time information.

5.4 Context: how close is the model to the achievable ceiling?

Because the data are simulated, we can compute the error of a **perfect predictor** that knows the exact noise-free expected time. On the same test set it still has MAE 3.80 min, RMSE 5.78 min and R^2 0.910, because of random noise and incidents. The fitted model (RMSE 6.18) is therefore within 0.40 min RMSE of the best any model could do here. This reference is not available for real data, where the ceiling is unknown and usually lower.

5.5 Robustness checks

- **Time-ordered split.** Training on the first 80% of the year and testing on the final 20% gave MAE 4.11, RMSE 5.99, R^2 0.903, similar to the random split. (This is expected since the simulator has no drift; on real data, a time-ordered split is the more demanding test and is recommended.)
- **Error by segment** (mean absolute error, minutes): clear 3.85, rain 4.84, storm 5.49; bike 4.56, van 3.90, truck 6.38; peak 4.46 vs off-peak 3.88. The model is least accurate in storms, for trucks (only 1.4% of the data, so few examples) and at peak times, exactly where operational risk is highest, so alerts should carry a safety buffer.

- **Safety buffer.** The 90th percentile of out-of-fold residuals on the training data is 6.29 min. Adding it to a prediction covered 89.5% of hold-out deliveries, close to the intended 90%, which confirms the buffer is well calibrated. It is used in the alert module.

5.6 Which inputs matter?

Table 3. Permutation importance on the hold-out set (increase in RMSE when a feature is shuffled)

Rank	Feature	RMSE increase (min)
1	distance_km	19.05
2	vehicle_type	5.53
3	stops_on_route	2.93
4	weather	1.82
5	traffic_index	1.41
6	priority	0.38
7	vehicle_load_pct	0.18
8	package_weight_kg	0.09

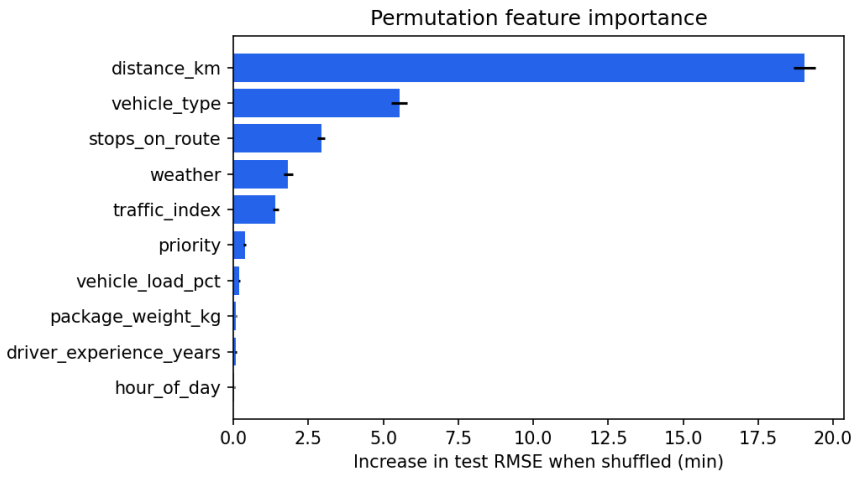


Figure 4. Permutation feature importance (top 10).

Distance dominates, followed by vehicle type, number of stops, weather and the traffic index. Hour of day, day of week and zone show near-zero permutation importance. This does **not** mean they are irrelevant: in the data, their effect operates through the traffic index (the congestion profile depends on hour, zone and weekday), so once traffic is known they add little extra information. Permutation importance understates the value of correlated features; the practical lesson is that a reliable real-time **traffic index is the key input** to collect.

6. Optimization Strategies

Prediction alone does not improve operations; decisions do. Four modules turn the model's output into actions, ordered by when they occur in the daily workflow:

Stage	Module	Model output used	Decision
Planning (day before / morning)	6.1 Dispatch planning	Predicted time by hour	When to release flexible deliveries

Stage	Module	Model output used	Decision
Pre-dispatch	6.2 Traffic-risk alerts	Predicted time + safety buffer vs promised ETA	Warn, re-quote or re-assign at-risk deliveries
Dispatch	6.3 Vehicle allocation	Predicted time for each parcel × vehicle	Which parcel goes on which vehicle
On the road	6.4 Route planning	Learned minutes-per-km versus congestion	Order of stops on a run

How the optimisation results are evaluated

Decisions are made using the **model's predictions**, but outcomes are scored using the simulator's noise-free ground truth (`expected_time`). This avoids grading the model with its own opinion. The reported gains are **simulated gains** under assumptions stated in each subsection; they show the method works on this synthetic system and are not savings measured at any real company.

6.1 Dispatch planning: shift flexible deliveries away from congestion

Idea. Express parcels must leave immediately, but many standard deliveries are booked into time slots. If a standard delivery planned for a congested hour can leave up to two hours later at a lower predicted time, vehicle time is saved. The model-predicted time for a reference delivery (8 km, 5 stops, van, clear weather) is lowest at 07:00 (32.9 min) and highest at 18:00 (39.4 min): a spread of 6.5 minutes for identical work.

Table 4. Predicted delivery time of the reference delivery by dispatch hour

Dispatch hour	Model prediction (min)	Simulator truth (min)
07:00	32.9	32.8
09:00	36.7	36.1
12:00	33.2	32.7
15:00	33.6	32.6
18:00	39.4	36.8
20:00	34.9	33.5

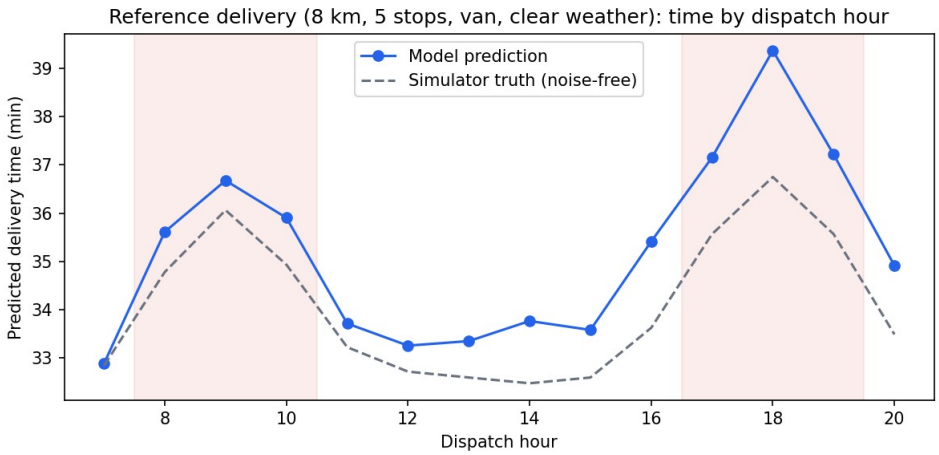


Figure 5. Predicted time by dispatch hour (shaded bands = morning and evening peaks).

Method. For every standard-priority delivery dispatched in a peak hour (08–10, 17–19), the model predicts the time for departures 0, +1 and +2 hours later (traffic index shifted by the hourly profile). A delivery is moved only if the predicted saving is at least 2 minutes (to avoid chasing noise).

Code 3. Choosing a dispatch shift (src/optimize.py, simplified)

```
for s in range(MAX_SHIFT + 1):                        # candidate delays 0, 1, 2 h
    pred_s.append(model.predict(shifted_frame(s)))   # decisions use the MODEL
    true_s.append(expected_time(shifted_clean(s)))   # outcomes use SIMULATOR truth
gain_pred = pred_s[0] - pred_s.min(axis=0)
chosen = np.where(gain_pred >= MIN_GAIN, pred_s.argmin(axis=0), 0)
realised_saving = true_s[0] - true_s[chosen, np.arange(n)]
```

Result (hold-out deliveries)	Value
Standard deliveries dispatched at peak	459 (38.2% of all test deliveries)
Shifted by the model	180 (39.2%), by 1.81 h on average
True minutes saved per shifted delivery	4.00 min
Total on-road vehicle minutes saved	719.8 min (3.6% of the group's on-road time)
Share of the perfect-information saving captured	75.2%

Caveats. The saving is in **vehicle time and cost**, not customer waiting time: shifted deliveries arrive later in the day, so this only suits deliveries booked into flexible slots. Express parcels and tight slots are never moved. In practice the shift window should be set by the customer's slot, not fixed at two hours.

6.2 Traffic-risk alerts

Idea. Before a vehicle leaves, compare the predicted delivery time with the ETA already promised to the customer and raise a colour-coded alert. This turns the forecast into an actionable early warning that gives dispatchers time to re-quote the customer, change vehicle or re-sequence the run.

Level	Rule	Action
Red	Predicted time > promised ETA	Likely late: re-quote the customer, upgrade vehicle or re-assign
Amber	Predicted time + 6.3 min buffer > promised ETA	At risk: monitor, avoid adding stops to this run
Green	Otherwise	Proceed normally

Code 4. Alert logic (src/optimize.py)

```
level = np.where(pred > promised, 'Red',
         np.where(pred + buffer_q90 > promised, 'Amber', 'Green'))
late  = actual > promised                        # ground truth, used only for scoring
```

Table 5. Alert performance on the hold-out test set

Alert level	Deliveries	Share	Actual late rate
Red	257	21.4%	76.3%
Amber	274	22.8%	20.8%
Green	669	55.8%	3.1%

Alert level	Deliveries	Share	Actual late rate
All deliveries	1,200	100%	22.8%

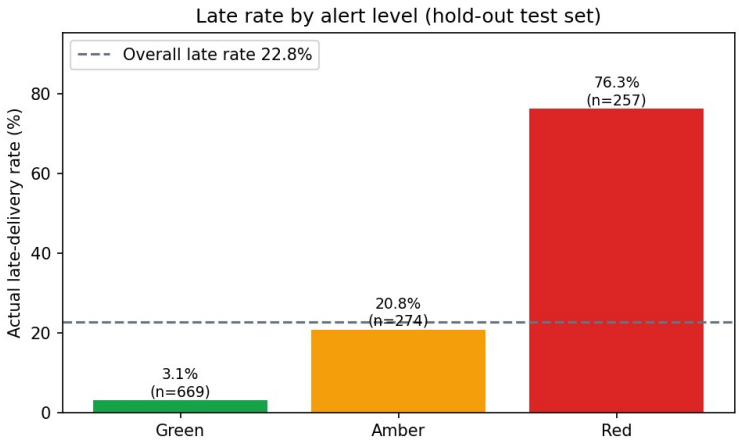


Figure 6. Actual late-delivery rate for each alert level.

- **The alerts separate risk well:** Red deliveries are late 76.3% of the time versus 3.1% for Green, a 24.6-fold difference.
- **Coverage:** Red or Amber captured 92.3% of all late deliveries (precision 47.6%, flagging 44.2% of deliveries). Red alone has precision 76.3% and recall 71.5%. Operations can choose the trade-off: act on Red only to limit workload, or on Red + Amber to catch almost every late delivery.
- **Versus a simple traffic rule:** a "traffic index ≥ 7" alert flags only 4.1% of deliveries and catches just 9.9% of late ones. Lateness is mostly caused by combinations (distance, stops, weather, vehicle) that a single-threshold rule misses.
- **Why deliveries were flagged:** adverse weather 40.7%, peak hour 27.7%, long distance or many stops 23.2%, very high traffic 8.5%. Showing a reason tag next to each alert helps dispatchers trust and act on it.

6.3 Vehicle allocation

Idea. Choosing a vehicle changes both the delivery time and the cost. Bikes are cheap per minute but slow and weather-sensitive; vans are the workhorse; trucks are expensive but needed for heavy parcels. For each dispatch wave, the model predicts the time of every parcel on every feasible vehicle and an assignment algorithm picks the cheapest overall plan.

Problem. Minimise total cost = fixed trip cost + cost per minute × predicted time + late penalty × minutes beyond the promised ETA, subject to one parcel per vehicle and feasibility limits (bike: parcel ≤ 10 kg and ≤ 15 km; van: ≤ 150 kg; truck: any). This is a classic assignment problem solved exactly with the Hungarian algorithm (Kuhn, 1955; `scipy.optimize.linear_sum_assignment`). Cost figures are **assumptions**: bike ₹4/min + ₹20 per trip, van ₹7/min + ₹60, truck ₹12/min + ₹150, late penalty ₹10/min.

Code 5. Cost-minimising assignment (src/optimize.py, simplified)

```
# C_pred[i, j] = fixed[v] + rate[v]*pred_time + late_penalty*max(0, pred_time -
promised)
#                (inf if parcel i is infeasible on vehicle j)
rows, cols = linear_sum_assignment(C_pred)   # optimal one-to-one assignment
```

```
# Baseline = today's rule: bike for small parcels, van for others, truck only if >150
kg,
# first free vehicle in that preference order.
```

Experiment. 200 dispatch waves, each with 20 parcels drawn from the hold-out set and a fleet of 24 vehicles (8 bikes, 12 vans, 4 trucks). Each plan is scored with the simulator's true times.

Per dispatch wave (avg of 200)	Rule-based	Model-guided	Perfect-information bound
Total cost (₹)	6,037	5,712	5,692
Vehicle minutes	839.3	807.4	807.7
Late parcels (of 20)	4.15	2.39	2.17

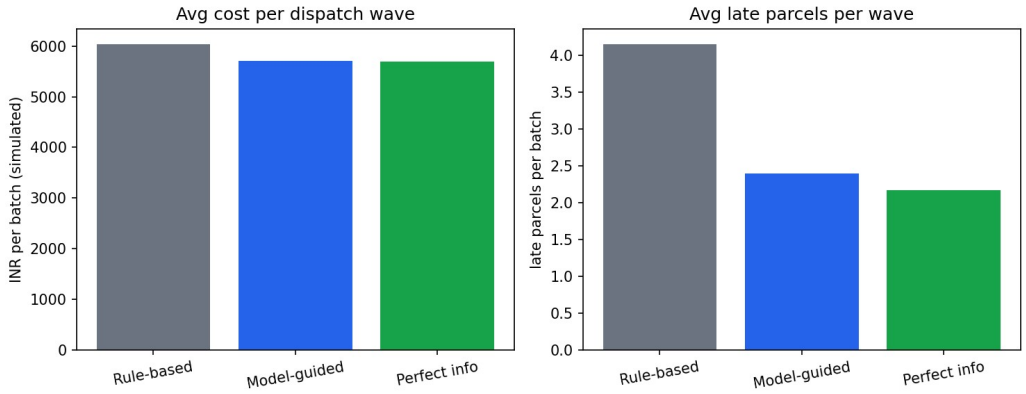


Figure 7. Average cost (left) and late parcels (right) per dispatch wave for three allocation strategies.

- Model-guided allocation reduces simulated cost by **5.4%** and late parcels by **42.4%**, and is cheaper than the rule-based plan in 96.5% of waves.
- It captures **94.2%** of the saving that would be possible with perfect knowledge of delivery times, showing that the model is accurate enough to support this decision.
- The main behaviour change: the optimiser sends parcels that are at risk of lateness (long distance, bad weather, many stops) on faster vehicles and keeps bikes for short, low-risk parcels, instead of defaulting to the cheapest feasible vehicle.

Caveats. Counterfactual predictions (the same parcel on a different vehicle) are only trusted inside the feasible region, where the training data has examples of that vehicle type; trucks have few samples. One-parcel-per-vehicle is a simplification of real multi-drop runs, and cost parameters must be replaced with actual company rates.

6.4 Route-planning improvements

Idea. Within a vehicle's run, the order of stops determines total driving time. Two improvements were tested on 300 simulated runs of 8 stops in a 20 × 20 km city with a congested core and two random hotspots (for example accidents or events):

- **A. Nearest neighbour on distance**: the common greedy baseline.
- **B. Nearest neighbour + 2-opt on distance**: an improvement heuristic that repeatedly reverses route segments when that shortens the tour (Croes, 1958).
- **C. Nearest neighbour + 2-opt on traffic-aware travel times**: the same optimiser, but the cost of each leg comes from the model. The model's **learned minutes-per-km** at the congestion

level of the two stops is used: pk(t) = [pred(15 km, t) − pred(5 km, t)] ÷ 10, which cancels fixed loading and waiting time. The curve is smoothed to be non-decreasing (isotonic regression) because congestion cannot speed traffic up.

The model's learned curve is close to the truth at moderate congestion (2.22 and 2.81 min/km at index 0 and 5, versus 2.14 and 2.89 true) but underestimates it at the extreme end (3.44 vs 4.29 at index 10), because gridlock rows are rare in the training data.

Route strategy	Average true route time (min)	vs A
A. Nearest neighbour (distance)	198.3	-
B. 2-opt (distance)	184.6	-6.9%
C. 2-opt (traffic-aware, model)	184.5	-7.0%
Oracle: 2-opt with perfect traffic knowledge	184.1	-7.2%

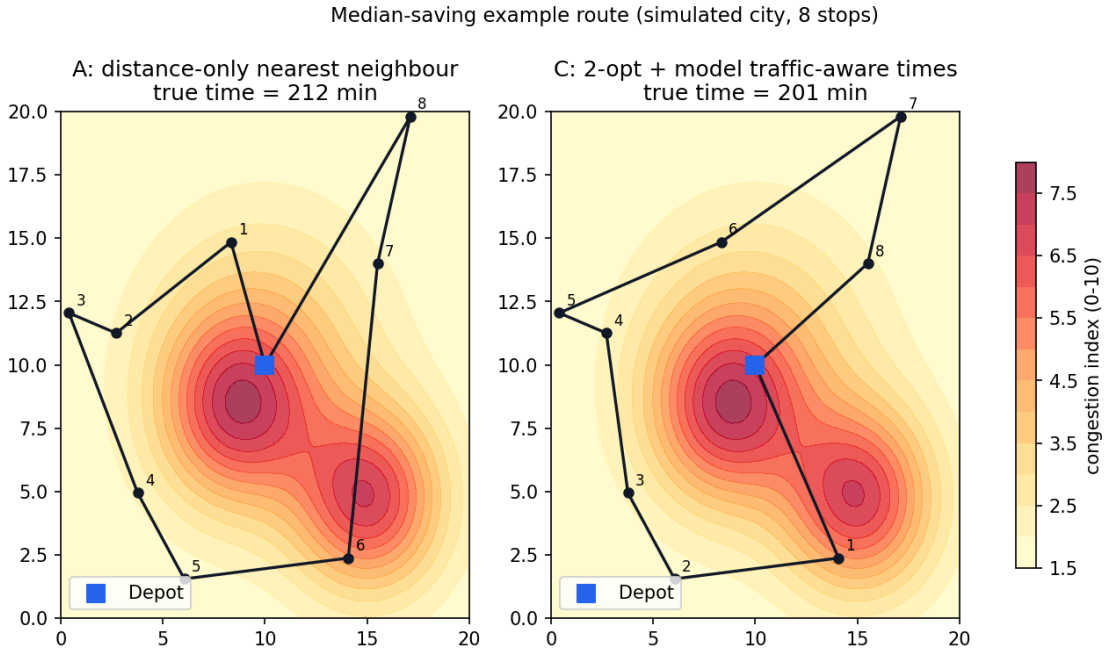


Figure 8. A median-saving example: distance-only route (left) versus the improved route (right) over the congestion map.

An honest finding

Replacing the greedy route by a 2-opt optimised route cut average route time by **6.9%** (13.8 minutes per run), and the optimised route was faster in 80.7% of runs. However, **making the optimiser traffic-aware added only 0.03%** on top of 2-opt, and even perfect traffic knowledge would add only 0.23%. The reason is structural: when every stop must be visited and the vehicle returns to the depot, congestion mostly scales all legs similarly and cannot be avoided, so the geometry of the tour dominates. The example in Figure 8 shows this: the gain comes mainly from removing crossing legs, not from dodging the hotspots.

Practical conclusion. Invest first in a proper route optimiser (2-opt or a solver such as OR-Tools). Traffic-aware travel times become valuable when the problem offers real choices that congestion can influence, such as customer time windows, choosing which stops to put on which run, alternative road paths, or the departure time. That is why the traffic signal is used in modules 6.1 to 6.3, where it gave larger effects.

7. Recommendations and Implementation Roadmap

The recommendations below are ranked by the strength of the evidence in this study and by ease of adoption. Impact figures come from the simulation and must be re-measured on real data before being used for business cases.

#	Recommendation	Evidence (simulated)	KPI to track
1	Replace the rule-of-thumb ETA with the model's prediction for customer quotes	ETA error 9.92 → 4.15 min MAE	ETA MAE, on-time rate
2	Switch on Red / Amber pre-dispatch alerts with reason tags; start with Red only, add Amber once the team can handle the volume	Red late rate 76.3% vs Green 3.1%; Red+Amber catches 92.3% of late deliveries	Alert precision / recall, late rate, re-quote rate
3	Use model-guided vehicle allocation for each dispatch wave, with a human override	Cost -5.4%, late parcels -42.4%	Cost per delivery, late parcels per wave
4	Offer flexible delivery slots and release flexible deliveries outside the evening and morning peaks	On-road minutes of eligible deliveries -3.6%	Vehicle-hours per delivery, slot uptake
5	Adopt a route optimiser (2-opt or solver) for multi-stop runs before investing in traffic-aware costs	Route time -6.9%	Km and minutes per run
6	Invest in a reliable live traffic index and weather feed	Traffic and weather are among the top inputs; the traffic index also drives the alerts	Data completeness, feed latency

7.1 Implementation roadmap

Phase	Actions
0-30 days: foundation	Collect real dispatch, GPS and delivery timestamps; define the target precisely; build the data pipeline; replicate this study on real data using the same scripts.
30-90 days: pilot	Run the model in shadow mode (predict but do not act) on one depot; compare predictions with actual times; calibrate the alert buffer; test Red alerts with dispatchers.
90+ days: scale	Enable allocation support and flexible-slot release; add route optimisation; extend to more depots.
Ongoing: monitoring	Track weekly MAE and bias by weather, vehicle and hour; retrain monthly or when MAE degrades by more than a set threshold; log every override to learn from dispatcher judgement.

8. Limitations and Threats to Validity

- **Simulated data.** The most important limitation. Performance on real data will almost certainly be lower and will need re-validation. The model has learned the simulator's rules, not the behaviour of a real city.
- **Optimistic evaluation.** Optimisation gains are scored against the same simulator that generated the training data. They demonstrate the method, not a measured business saving.
- **Assumed costs.** All cost rates, fixed trip costs and late penalties in Section 6.3 are illustrative assumptions.
- **No drift.** The simulated process is stationary; real logistics data changes with seasons, road works and demand, so monitoring and retraining are essential.

- **Simplified decisions.** One parcel per vehicle in the allocation experiment, a fixed 20 × 20 km city and simple hotspot congestion in the routing experiment, and a fixed flexibility window in dispatch planning.
- **Sparse segments.** Trucks and storms have few observations and larger errors.
- **Correlated features.** Permutation importance understates the role of hour, zone and weekday, whose effects pass through the traffic index.
- **Fairness and workload.** In a real deployment, alerts and allocation changes affect drivers' workloads; they should be introduced with human oversight and clear override rules.

9. Reproducibility and Code Verification

Every random process uses a fixed seed (42), and all preprocessing happens inside a scikit-learn Pipeline, so each run produces the same dataset, models, metrics and figures. Two scripts are provided:

Commands

```
pip install -r requirements.txt
python src/run_pipeline.py            # data -> models -> optimisation (about 2-5
minutes)
python verify_reproducibility.py      # re-runs everything in a temp folder and
compares
```

`verify_reproducibility.py` re-runs the **entire** pipeline in a temporary folder and compares every headline number in the three result files with the committed results (tolerance 0.001), and requires the regenerated dataset CSV to be byte-identical.

Verification result at the time of writing
PASS. The full pipeline was re-run from scratch on 2026-10-18 (Python 3.12.3, numpy 2.4.4, pandas 3.0.2, scikit-learn 1.8.0). The regenerated dataset was byte-identical and all 143 compared result values matched the committed results (mismatches: 0).

The report itself is generated from the result files by `report/build_report.js`, so the numbers in this document cannot drift from the code output.

10. Conclusion

This project defined a clear logistics forecasting problem, built a simulated but realistic dataset, and compared six regression techniques with consistent cross-validation, tuning and hold-out testing. Gradient boosting was the best model (test MAE 4.15 min, RMSE 6.18 min, R^2 0.897), outperforming linear regression by about 13.7% in RMSE and clearly beating the rule-of-thumb ETA.

The model's value was shown through decisions rather than accuracy alone. Pre-dispatch alerts separate high-risk from low-risk deliveries, model-guided vehicle allocation reduces simulated cost and lateness, dispatch timing can trim on-road time for flexible deliveries, and route optimisation shortens runs, although traffic-aware routing itself added little. Reporting that negative result, together with the explicit limits of simulated data, is part of keeping the conclusions honest.

The natural next step is to replicate the study on real operational data, where the same scripts, evaluation protocol and monitoring plan can be reused.

References

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281-305.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

Croes, G. A. (1958). A method for solving traveling-salesman problems. Operations Research, 6(6), 791-812.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer.

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, 2(1-2), 83-97.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Appendix A: Repository Structure

```
logistics-delivery-time-prediction/
|-- README.md                      project overview and how to run
|-- requirements.txt               Python dependencies
|-- verify_reproducibility.py      re-runs pipeline and checks results match
|-- data/
|   `-- delivery_data.csv          SIMULATED dataset (6,000 deliveries)
|-- src/
|   |-- common.py                  paths, seed, features, split
|   |-- simulator.py               data-generating process (digital twin)
|   |-- generate_data.py           step 1: create dataset + EDA figure
|   |-- train_models.py            step 2: CV, tuning, evaluation
|   |-- optimize.py                step 3: dispatch, alerts, allocation, routes
|   `-- run_pipeline.py            runs steps 1-3
|-- results/                       metrics (JSON/CSV), model, figures/
`-- report/
    |-- build_report.js            generates this report from results/
    `-- Logistics_Predictive_Modeling_Report.docx
```

Appendix B: Tuned Hyper-parameters

Selected by randomized search (up to 8 candidates, 3-fold inner cross-validation, RMSE criterion) on the training set.

Model	Selected hyper-parameters
Ridge Regression	alpha = 10
Decision Tree	min_samples_leaf = 10; max_depth = 12
Random Forest	n_estimators = 100; min_samples_leaf = 3; max_features = 0.6; max_depth = 15
Gradient Boosting	subsample = 0.7; n_estimators = 350; min_samples_leaf = 10; max_depth = 4; learning_rate = 0.05